

Optimization

Overview of Methods

John Snyder

STAT 9340: Spring 2023

Modeling

In a supervised learning problem, we have a p_x dimensional input vector $\mathbf{x}_i$, with an associated target vector $\mathbf{y}_i$ for each of the $i = 1, \dots, n$ data points. We create a model f with a set of p_θ parameters θ as

$$f(\mathbf{x}_i; \theta)$$

for the purpose of predicting $\mathbf{y}_i$ for input $\mathbf{x}_i$.

Our target vector $\mathbf{y}_i$ could (for example) be:

- a. A single continuous output value.
- b. Membership in one of 10 classes.

If we would like our model to generate predictions that are close to the true $\mathbf{y}_i$, we need to find a suitable set of values for θ .

For some models, we have simple closed form solutions, but for complex structures, we often cannot easily find the best functional form.

Section 1

Optimization Strategies

Finding θ .

Typically to find θ a *loss function* which somehow describes the distance between $\mathbf{y}_i$ and $f(\mathbf{x}_i; \theta)$ must be constructed.

In example a, we would likely use the squared error loss

$$Q(f(\mathbf{x}); y; \theta) = \sum_{i=1}^n (y_i - f(\mathbf{x}_i; \theta))^2.$$

In example b, we would use *Cross Entropy Loss*

$$Q(f(\mathbf{x}); y; \theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^{10} y_{i,c} \log(f(\mathbf{x}_i; \theta)_c).$$

In **either** case, we seek a set of model parameters that minimize the loss function Q :

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}(Q(f(\mathbf{x}); y; \theta))$$

A Note on Classification:

Just like with Generalized Linear Models, we must respect the fact that we are predicting a probability $p \in (0, 1)$ so we should ensure that whatever is coming out of our model maps to that interval.

- Typically, we will let models learn on whatever space they want and then squeeze them into $(0, 1)$ with the *softmax function*

$$p(y_i = k \mid \mathbf{x}_i, \boldsymbol{\theta}) = \frac{\exp(\boldsymbol{\theta}'_k \mathbf{x}_i)}{\sum_{\ell=1}^K \exp(\boldsymbol{\theta}'_{\ell} \mathbf{x}_i)}.$$

This function takes a linear outputs for each class and scales them to be in $(0, 1)$ so that their sum is 1.

- In the binary case, this would just be the *sigmoid*.

Optimization: Finding a suitable θ

GOAL: Find a set of θ that minimize the loss function Q .

In the derivation of many statistical methods, we can use calculus to derive closed form solutions that guarantee some forms of optimality of simple calculations.

The scale of most problems in this space prohibits us from explicitly calculating a full solution, so we turn to optimization algorithms.

- Idea: Start with a guess and iterate, using the gradient as a guide.

In ML applications, this loss function will be referred to as the *objective function*

Optimization(cont)

We will iteratively update the current estimate of θ at step j (denoted as $\hat{\theta}^{(j)}$) according to:

$$\hat{\theta}^{(j+1)} = \hat{\theta}^{(j)} - \epsilon_j \mathbf{A}_j \frac{\partial Q(\hat{\theta}^{(j)})}{\partial \theta}$$

Here we have:

- $\frac{\partial Q(\hat{\theta}^{(j)})}{\partial \theta}$, the gradient of the objective function $Q(\theta)$ at $\hat{\theta}$
 - ▶ Tells us direction each parameter should move to reduce loss.
 - ▶ Also denoted as $\nabla_{\theta} Q(\hat{\theta}^{(j)})$
- ϵ_j , the “learning rate,” which allows us to stretch or shrink the magnitude of movement.
- $\mathbf{A}_j$ a positive definite matrix
 - ▶ Typically the inverse of the squared Jacobian(Gauss-Newton) or Hessian(Newton-Raphson)
 - ▶ Allows the optimization to have a deeper understanding of the surface being optimized.

Gradient Descent

In practice, calculations to obtain $\mathbf{A}_j$ are computationally expensive (matrix inversions) and therefore not feasible.

- Gradient Descent will assume that $\mathbf{A}_j \equiv I_p$ and we will thus have

$$\hat{\boldsymbol{\theta}}^{(j+1)} = \hat{\boldsymbol{\theta}}^{(j)} - \epsilon_j \frac{\partial Q\left(\hat{\boldsymbol{\theta}}^{(j)}\right)}{\partial \boldsymbol{\theta}}.$$

The steps here are straightforward:

- 1 Evaluate the gradient of the loss function over the data at the current parameter estimate values.
- 2 Multiply the vector of gradients by ϵ_j .
- 3 Adjust parameter estimates and repeat.

Introducing Stochasticity

Unfortunately for large datasets and/or large parameter spaces the evaluation of $\frac{\partial Q(\hat{\theta}^{(j)})}{\partial \theta}$ can not be done all at once.

- Solution: Update parameters incrementally by presenting the gradient descent algorithm with random small samples of data for its updates.

This approach leads to **Stochastic** gradient descent (SGD)

- Leads to faster and more stable convergence with noisy data.

Overall, SGD is a very effective optimization algorithm for deep learning and is widely used in practice.

The following algorithms, unless noted, are presented from (Goodfellow et al. 2016, Chap. 8)

Stochastic Gradient Descent

Stochastic gradient descent (SGD) update at training iteration j

Require: Learning Rate ϵ_j

Require: Initial Parameter values $\theta^{(1)}$

```
1 while Stopping criterion not met do
2   |   Sample a minibatch of  $m$  examples from the training set
      |    $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  with corresponding targets  $\mathbf{y}_i$ ;
3   |   Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i Q\left(f\left(\mathbf{x}^{(i)}; \theta\right), \mathbf{y}^{(i)}\right)$ ;
4   |   Apply update:  $\theta^{(j+1)} \leftarrow \theta^{(j)} - \epsilon_j \hat{\mathbf{g}}$ ;
5 end
```

- The learning rate ϵ_j is critical and is often the main hyperparameter to be tweaked in many applications.
- To get optimal performance one can “schedule” adjustments so that a model learns quickly at first, and slower later.

Gradient Descent Learning Rate Adjustments

There are many approaches to adjusting the optimization strategy of ML models as they are being trained.

- Momentum: Add a tuning parameter which factors in the previous step's gradient to the current gradient estimate.
 - ▶ Issue: Updates all parameters using the same value
- Adagrad: adapts the learning rates to each parameter by scaling them inversely proportional to the sum of all historical squared gradients
 - ▶ Issue: Learning rates may decay to 0.
- RMSProp: Adagrad, but use a weighted moving average of squared gradients rather than the sum of all historical.
- AdaDelta: Adagrad, but use an exponential decaying average of square updates.
- Adam: Combines the best of both momentum based updates bringing in historical gradients and adaptive approaches which allow flexibility in each parameter update.
- Others based on adjustments to Adam (AdamW, etc)

Stochastic Gradient Descent with Momentum

IDEA: Add a prop. of all previous step's gradient to the current update

- The fraction is a parameter that is tuned with parameter α .
- Often results in faster convergence in flat regions and slower convergence in steep regions.

Require: Learning rate ϵ

Require: Momentum parameter α

Require: Initial velocity $\mathbf{v}$

Require: Initial Parameter values $\boldsymbol{\theta}^{(1)}$

```
1 while Stopping criterion not met do
2   Sample a minibatch of  $m$  examples from the training set
   { $\mathbf{x}_1, \dots, \mathbf{x}_m$ } with corresponding targets  $\mathbf{y}_i$ ;
3   Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i Q(f(\mathbf{x}_i; \boldsymbol{\theta}), \mathbf{y}_i)$ ;
4   Compute velocity update:  $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \hat{\mathbf{g}}$ ;
5   Apply update:  $\boldsymbol{\theta}^{(j+1)} \leftarrow \boldsymbol{\theta}^{(j)} + \mathbf{v}$ ;
6 end
```

The AdaGrad Algorithm

IDEA: Adjust parameters individually.

- Makes convergence much less dependent on the overall learning rate.

Require: Global learning rate ϵ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Require: Initial Parameter values $\theta^{(1)}$

Init: Gradient accumulation variable $r = 0$

1 **while** *Stopping criterion not met* **do**

2 Sample a minibatch of m examples from the training set
 $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ with corresponding targets $\mathbf{y}_i$;

3 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i Q(f(\mathbf{x}_i; \theta), \mathbf{y}_i)$;

4 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$;

5 Compute update: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$. (Division and square root
 applied element-wise);

6 Apply update: $\theta^{(j+1)} \leftarrow \theta^{(j)} + \Delta\theta$;

7 **end**

The RMSProp Algorithm

IDEA: AdaGrad, but using an exponentially weighted moving average in $\mathbf{r}$.

Require: Global learning rate ϵ

Require: Decay rate ρ

Require: Small constant δ , perhaps 10^{-6} , for numerical stability

Require: Initial Parameter values $\theta^{(1)}$

Init: Gradient accumulation variable $\mathbf{r} = 0$

```
1 while Stopping criterion not met do
2   Sample a minibatch of  $m$  examples from the training set
    $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  with corresponding targets  $\mathbf{y}_i$ ;
3   Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i Q(f(\mathbf{x}_i; \theta), \mathbf{y}_i)$ ;
4   Accumulate squared gradient:  $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$  (where  $\odot$  is
   element-wise multiplication);
5   Compute update:  $\Delta \theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$ . (Division and square root
   applied element-wise);
6   Apply update:  $\theta^{(j+1)} \leftarrow \theta^{(j)} + \Delta \theta$ ;
7 end
```

The Adam Algorithm

Adaptive Moment Estimation

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $(0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ , perhaps 10^{-8} , for numerical stability

Require: Initial Parameter values $\theta^{(1)}$

Init: Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Init: Initialize time step $t = 0$

```
1 while Stopping criterion not met do
2     Sample a minibatch of  $m$  examples from the training set  $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  with
       corresponding targets  $\mathbf{y}_i$ ;
3     Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i Q(f(\mathbf{x}_i; \theta), \mathbf{y}_i)$ ;
4     Update biased first moment estimate:  $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \hat{\mathbf{g}}$ ;
5     Update biased second moment estimate:  $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$ ;
6     Correct bias in first moment:  $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$ ;
7     Correct bias in second moment:  $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$ ;
8     Compute update:  $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$  (operations applied element-wise);
9     Apply update:  $\theta^{(j+1)} \leftarrow \theta^{(j)} + \Delta \theta$ ;
10 end
```

Regularizing Models and Constrained Optimization

Recall when we regularize models we essentially would like to minimize the loss $Q(\boldsymbol{\theta})$ subject to *constraints* that shrink parameters towards 0.

- Ridge regression - $\underset{\boldsymbol{\theta}}{\operatorname{argmin}} Q(\boldsymbol{\theta})$ under the constraint $\sum_{j=1}^p \theta_j^2 \leq s$
- Lasso regression - $\underset{\boldsymbol{\theta}}{\operatorname{argmin}} Q(\boldsymbol{\theta})$ under the constraint $\sum_{j=1}^p |\theta_j| \leq s$

These can also be formulated as unconstrained minimization problems.

- Ridge regression - $\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left\{ Q(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p \theta_j^2 \right\}$
 - ▶ L_2 Penalty
- Lasso regression - $\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left\{ Q(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p |\theta_j| \right\}$
 - ▶ L_1 Penalty

Solving a Constrained Optimization Problem

- When constraints are equalities, constrained optimization problems can be solved with Lagrange Multipliers
 - ▶ Create a system of equations derived from where the constraint and function are tangent to one another
- Occasionally, we can transform the constrained parameter space so that the transformed parameters are unconstrained.
 - ▶ e.g. Minimizing $f(x_1, x_2)$ under $\sqrt{x_1 + x_2} = 1$ can be accomplished by minimizing $g(\theta) = f(\cos \theta, \sin \theta)$ and transforming the result back.
- Early attempts at solving this type of problem involved executing steps of gradient descent only if proposals satisfied the constraints.
- The *Karush-Kuhn-Tucker (KKT)* approach provides a general solution that handles equality and inequality constraints together
 - ▶ Works off the concept of *The Generalized Lagrangian*

The Generalized Lagrangian (S4.4 in DL book)

We consider the set of feasible parameter values

$$\mathcal{S} = \left\{ \boldsymbol{\theta} \mid \forall i, g^{(i)}(\boldsymbol{\theta}) = 0 \text{ and } \forall j, h^{(j)}(\boldsymbol{\theta}) \leq 0 \right\}$$

So $g^{(i)}$ are equality constraints and $h^{(j)}$ inequality constraints. We further introduce the *KKT Multipliers* λ_i and α_j for each constraint and present the generalized Lagrangian as

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\alpha}) = Q(\boldsymbol{\theta}) + \sum_i \lambda_i g^{(i)}(\boldsymbol{\theta}) + \sum_j \alpha_j h^{(j)}(\boldsymbol{\theta}).$$

Just like looking at Lasso/Ridge, the above is an expression that can be optimized in an unconstrained fashion! This is because one can show:

$$\min_{\boldsymbol{\theta}} \max_{\boldsymbol{\lambda}} \max_{\boldsymbol{\alpha}, \boldsymbol{\alpha} \geq 0} L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\alpha})$$

has the same optimal objective function value and set of optimal points $\boldsymbol{\theta}$ as

$$\min_{\boldsymbol{\theta} \in \mathcal{S}} Q(\boldsymbol{\theta}).$$

Constrained Linear Least Squares(S4.5 in DL)

In multiple regression we want to find the slopes θ that minimize the SSE

$$Q(\theta) = \frac{1}{2} \|\mathbf{X}\theta - \mathbf{y}\|_2^2,$$

where $\|\cdot\|_2$ is the L_2 norm. The gradient is then easily shown to be

$$\nabla_{\theta} Q(\theta) = \mathbf{X}'\mathbf{X}\theta - \mathbf{X}'\mathbf{y}.$$

Setting this to $\mathbf{0}$ yields the normal equations $\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Say instead we would like to perform ridge regression by adding the constraint

$$\sum_{j=1}^p \theta_j^2 = \theta^T \theta \leq s.$$

Constrained Linear Least Squares(S4.5 in DL)

The inequality constraint $\theta' \theta \leq s$ leads to the generalized Lagrangian

$$L(\theta, \nu) = Q(\theta) + \alpha (\theta' \theta - s)$$

and the solution is given by

$$\min_{\theta} \max_{\alpha, \alpha \geq 0} L(\theta, \alpha)$$

We can use a pseudoinverse to find a general solution to the unconstrained problem, but it may not satisfy the constraint.

Constrained Linear Least Squares(S4.5 in DL)

If the general solution is not feasible, we have to consider a so-called active solution, i.e., $h(\boldsymbol{\theta}) = 0$ for some $\boldsymbol{\theta}$. Note,

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \alpha) = \mathbf{X}'\mathbf{X}\boldsymbol{\theta} - \mathbf{X}'\mathbf{y} + 2\alpha\boldsymbol{\theta} = 0,$$

which, if α is known, gives a solution

$$\boldsymbol{\theta} = (\mathbf{X}'\mathbf{X} + 2\alpha\mathbf{I})^{-1} \mathbf{X}'\mathbf{y}$$

but where α must *be chosen to* allow $\boldsymbol{\theta}$ to satisfy the norm constraint. We do this by performing *gradient ascent* on the above with respect to α and solving for $\boldsymbol{\theta}$. Note,

$$\frac{\partial L(\boldsymbol{\theta}, \alpha)}{\partial \alpha} = \boldsymbol{\theta}'\boldsymbol{\theta} - s,$$

so when $\boldsymbol{\theta}'\boldsymbol{\theta} > s$ this gradient is positive, making α bigger in the gradient ascent, which then makes $\boldsymbol{\theta}$ smaller (i.e., gets it closer to satisfying the constraint).

Weight Decay Arising from L_2 Normalization

When we add in an L_2 penalty

$$Q_{reg}^{(j)}(\theta) = Q^{(j)}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

If we are executing SGD, we would have the parameters iterating according to

$$\begin{aligned}\theta^{(j+1)} &\leftarrow \theta^{(j)} - \epsilon \nabla Q_{reg}^{(j)}(\theta^{(j)}) \\ &= \theta^{(j)} - \epsilon \nabla Q^{(j)}(\theta) - \epsilon \lambda \theta^{(j)} \\ &= (1 - \epsilon \lambda) \theta^{(j)} - \epsilon \nabla Q^{(j)}(\theta)\end{aligned}$$

The factor $(1 - \epsilon \lambda)$ will be smaller than 1, thus this approach will tend to shrink parameters to 0. This is why the result of applying L_2 normalization to SGD is called *weight decay*.

The Adam algorithm with L_2 Regularization

Require: Step size ϵ (Suggested default: 0.001), $\lambda \in \mathbb{R}$

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $(0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ , perhaps 10^{-8} , for numerical stability

Require: Initial Parameter values $\theta^{(1)}$

Init: Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Init: Initialize time step $t = 0$ and schedule multiplier $\eta_{t=0} \in \mathbb{R}$

1 **while** *Stopping criterion not met* **do**

2 Sample a minibatch of m examples from the training set $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ with
corresponding targets $\mathbf{y}_i$;

3 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}_i; \theta), \mathbf{y}_i) + \lambda \theta^{(j-1)}$;

4 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \hat{\mathbf{g}}$;

5 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$;

6 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$;

7 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$;

8 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise);

9 Apply update: $\theta^{(j+1)} \leftarrow \theta^{(j)} + \Delta \theta$;

10 **end**

Adaptive Optimizers and L_2 regularization

Even though Adam was a great “default” choice of optimization strategy, applying L_2 regularization never yielded substantial performance gains.

- Circa ~2018, top models were using SGD with carefully tuned weight decay and learning rate schedules

Loshchilov & Hutter 2017 demonstrated that applying L_2 regularization to adaptive gradient optimization strategies (Adam, AdaGrad, etc) does *not* yield weight decay as it does in standard SGD

- Even when it works(in SGD), the learning rate ϵ is tightly coupled with the weight decay value λ .

The solution:

- “Decouple” the weight decay from the gradient adjustment

This approach often results in faster convergence and better out of sample generalization.

The AdamW Algorithm

Require: Step size ϵ (Suggested default: 0.001), $\lambda \in \mathbb{R}$

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $(0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ , perhaps 10^{-8} , for numerical stability

Require: Initial Parameter values $\theta^{(1)}$

Init: Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Init: Initialize time step $t = 0$ and schedule multiplier $\eta_{t=0} \in \mathbb{R}$

```
1 while Stopping criterion not met do
2     Sample a minibatch of  $m$  examples from the training set  $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  with
       corresponding targets  $\mathbf{y}_i$ ;
3     Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}_i; \theta), \mathbf{y}_i)$ ;
4     Update biased first moment estimate:  $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \hat{\mathbf{g}}$ ;
5     Update biased second moment estimate:  $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$ ;
6     Correct bias in first moment:  $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$ ;
7     Correct bias in second moment:  $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$ ;
8     Compute update:  $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}} - \lambda \theta^{(j-1)}$  (operations applied element-wise);
9     Apply update:  $\theta^{(j+1)} \leftarrow \theta^{(j)} + \Delta \theta$ ;
10 end
```

Learning Rate Scheduling

There are a many strategies we can use to control the learning rate ϵ in while performing optimization. This is especially useful for neural models that may get stuck in local minima.

- ❶ Fixed learning rate: In this strategy, the learning rate is kept fixed throughout the training process.
- ❷ Cyclical learning rate: In this strategy, the learning rate is not kept constant but is varied cyclically over time. One popular method for doing this is the “cyclical learning rate” (CLR) method, which involves periodically increasing and decreasing the learning rate over the course of training.
- ❸ Learning rate decay: In this strategy, the learning rate is decreased over time according to a pre-defined schedule.
- ❹ Warm restart: In this strategy, the learning rate is suddenly increased after a certain number of epochs, which can help the model escape from local minima and improve the overall convergence of the model. This can be combined with other learning rate adjustment techniques.

These are typically defined according to the total number of steps we will use to train a model (called “epochs”) so we will revisit these later.

Some final thoughts

Selecting batch sizes is often

- Larger batches give a more accurate estimation of the gradient but is often not worth the computational burden
- Extremely small batch sizes are often not used in practice because of computing architectures.

Learning rate adjustments are often trial and error

- Specific guidance on setting this value should be met with skepticism

The discussed approaches (and their derivatives) constitute the most popular approaches (at least with respect to deep modeling)

- The “best” choice is often more of an art than a science

General advice: Pick a few and become familiar with them.

- This is mostly because of hyperparameter tuning.

The Adam algorithm is fairly “set it and forget it.”

Section 2

Automatic Differentiation

Overview

Key: All we really need is to evaluate first derivatives *numerically*.

- Updating parameters through gradient descent a matter of evaluating derivatives of the gradient of the loss function at our current location.

There are 4 ways to do this (see Baydin et al. 2018):

- 1 Manually compute derivatives, code them, then substitute
 - Minor changes in model structure could completely change all calculations
- 2 Numerical differentiation/finite difference approximation
 - Application of the definition of the derivative
- 3 Symbolic differentiation via code
 - Most languages have libraries for this
- 4 Automatic differentiation(AD)
 - What is used in the big libraries.

Forward Difference

Given a scalar multivariate function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, we can approximate the gradients as

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

where $\mathbf{e}_i$ is the i -th unit vector and h is a small constant.

This is a straightforward approach, but has two key issues:

- 1 $\mathcal{O}(n)$ for n parameters
- 2 Numerical errors

Numerical errors are not actually a big problem, as oftentimes deep neural networks are fairly robust to errors in weights. This is sometimes used strategically (e.g. quantization) for various reasons. $\mathcal{O}(n)$ calculations is a problem though, as many times we have millions or billions of parameters to update in these frameworks.

Errors in Forward Difference approximation

Take the following simple example, evaluating $\left. \frac{\partial}{\partial x} \frac{1}{2}x^2 \right|_{x=3} = 3$

```
import matplotlib.pyplot as plt
import numpy as np

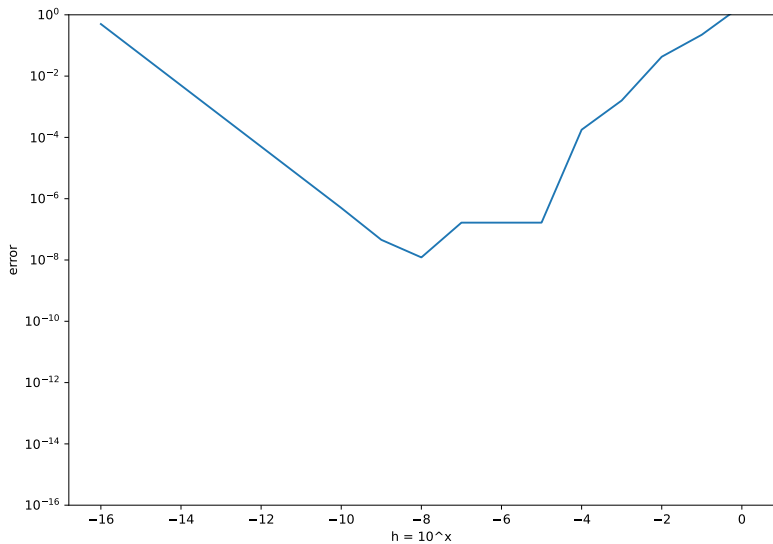
def f(x):
    return (1/2)*x**2

def finDiff(f, x, h, trueValue):
    return abs((f(x + h) - f(x))/h - trueValue)

err = [finDiff(f, x = 3, h = 10**(-exp), trueValue = 3) \
        for exp in range(17)]

_ = plt.plot(np.arange(-16,1), err)
_ = plt.yscale("log")
_ = plt.ylim((10**(-16), 1))
_ = plt.xlabel('h = 10^x')
_ = plt.ylabel('error')
```

Errors in Forward Difference approximation



Symbolic Differentiation via Software

We can express functions as data structures and software can store the “rules” that govern differential calculus applied to many common functions.

```
import sympy as sym
x = sym.Symbol('x') # declare symbolic variable explicitly
df_dx = sym.diff(0.5*x**2, x)
df_dx = sym.lambdify(x,df_dx)
df_dx(3)
```

```
## 3.0
```

```
f = expression(0.5*x^2) # Expression or formula
df_dx <- D(f, 'x')
eval(df_dx, list(x=3))
```

```
## [1] 3
```

This approach does provide a correct answer.

- The big drawback is *memory*.
- We don't need the symbolic form for our purposes, just the numerical result at the end
 - ▶ With nontrivial models, expressions would be *enormous*.
 - ★ This is called *expression swell*

Automatic differentiation (AD) overview

Automatic differentiation gives us the numerical results we need without the downfalls of symbolic differentiation and finite differencing. We decompose the mechanistic calculations into elementary steps that form an *evaluation trace*, and combine each step's derivative together through the chain rule.

It proceeds by:

- 1 Constructing a computation graph that represents the function to be differentiated.
- The graph consists of a set of nodes that represent the input variables and the operations performed on them, and a set of edges that represent the flow of data through the graph.
- 2 Given this graph, AD algorithms can then efficiently compute the derivatives of the function by applying the chain rule repeatedly to traverse the graph and compute the derivatives at each node.

Forward and Reverse mode AD

There are two main types of AD:

- ➊ Forward-mode AD computes the derivatives of the function with respect to **each** input variable, one at a time.
 - Works well with a small number of input variables.
 - $\mathcal{O}(n)$ calculations with n being the number of inputs
- ➋ Reverse-mode AD computes the derivatives of the function with respect to all of the input variables simultaneously.
 - Works well with a small number of output variables.
 - $\mathcal{O}(m)$ calculations with m being the number of inputs

Reverse-mode AD is generally more efficient and is the most commonly used form of AD in ML problems because we typically have millions of parameters to be updated but just a few(at most) outputs.

Constructing the Evaluation Trace

We consider a mapping $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that can have its intermediate steps be represented by the following variables:

- $v_{i-n} = x_i, i = 1, \dots, n$ are the input variables,
- $v_i, i = 1, \dots, l$ are the intermediate variables, of which
- $y_{m-i} = v_{l-i}, i = m - 1, \dots, 0$ are the output variables.

For example take the function $f(x_1, x_2) = y = x_1 x_2^2$

- $v_{-1} = x_1$
- $v_0 = x_2$
- $v_1 = v_0 \cdot v_0$
- $y = v_2 = v_{-1} \cdot v_1$

As we will see, we can use this flow of information, which has been broken down into chunks of elementary calculations, in order to numerically evaluate $\frac{\partial f}{\partial x_i}$.

A Real Example

Consider an example function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ from Griewank and Walther 2008:

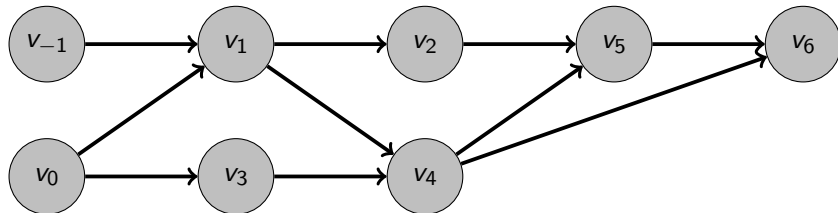
$$y = [\sin(x_1/x_2) + x_1/x_2 - \exp(x_2)] [x_1/x_2 - \exp(x_2)]$$

To stress numerical evaluation, let's say $x_1 = 8$ and $x_2 = 2$.

Intermediate Vars.	Expressions	Values
v_{-1}	x_1	8
v_0	x_2	2
v_1	v_{-1}/v_0	4
v_2	$\sin(v_1)$	-0.7568
v_3	$\exp(v_0)$	7.3891
v_4	$v_1 - v_3$	-3.3891
v_5	$v_2 + v_4$	-4.1459
v_6	$v_5 \cdot v_4$	14.0509

Real Example Computational Trace

$$y = [\sin(x_1/x_2) + x_1/x_2 - \exp(x_2)] [x_1/x_2 - \exp(x_2)]$$



Intermediate Vars.	Expressions	Values
v_{-1}	x_1	8
v_0	x_2	2
v_1	v_{-1}/v_0	4
v_2	$\sin(v_1)$	-0.7568
v_3	$\exp(v_0)$	7.3891
v_4	$v_1 - v_3$	-3.3891
v_5	$v_2 + v_4$	-4.1459
v_6	$v_5 \cdot v_4$	14.0509

Forward mode

Forward-mode automatic differentiation works by traversing the graph in a forward direction to compute the derivatives at each node.

In forward-mode AD, the derivatives of the function with respect to each input variable are computed one at a time. To compute the derivative of the function with respect to a particular input variable, the AD algorithm starts at the root node of the computation graph and applies the chain rule as it traverses the graph, keeping track of the partial derivatives at each node.

The values of the intermediate variables are sometimes called the primal trace, and the derivative values the tangent trace.

Example: Forward Mode (1/7)

In forward mode, we will fix the the *partial* part of the differential and traverse the graph until we finally obtain $\frac{\partial y}{\partial x_i}$. So for example to calculate the gradient of y w.r.t x_1 , with $x_1 = 8$ and $x_2 = 2$, we can progress through the intermediate steps and compute $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$ for **every** v_i .

First, note that

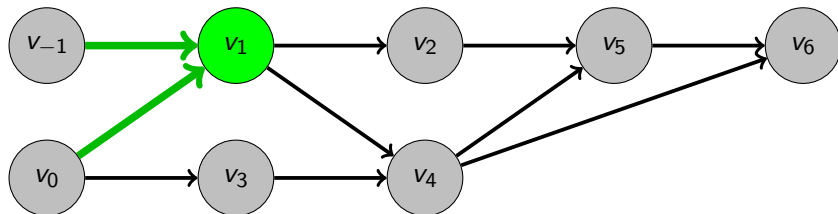
```
f = expression((sin(x1/x2) + x1/x2 - exp(x2))*(x1/x2 - exp(x2)))  
df_dx <- D(f, 'x1')  
eval(df_dx, list(x1=8, x2=2))
```

```
## [1] -2.65984
```

Obviously, to begin we have

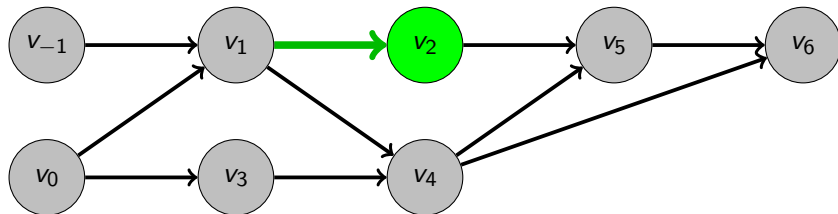
$$\dot{v}_{-1} = \frac{\partial x_1}{\partial x_1} = 1$$
$$\dot{v}_0 = \frac{\partial x_2}{\partial x_1} = 0$$

Example: Forward Mode (2/7)



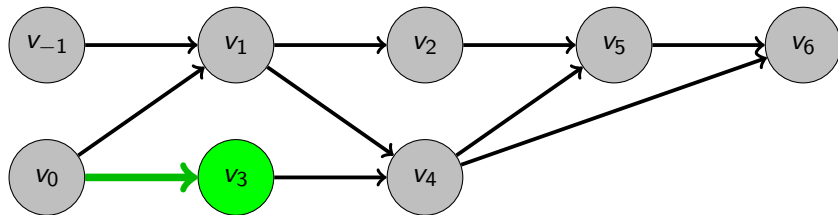
$$\dot{v}_1 = \frac{\partial (v_{-1}/v_0)}{\partial x_1} = \dot{v}_{-1} (v_0^{-1}) + \dot{v}_0 (-v_{-1}v_0^{-2}) = 1/2 + 0 = 1/2$$

Example: Forward Mode (3/7)



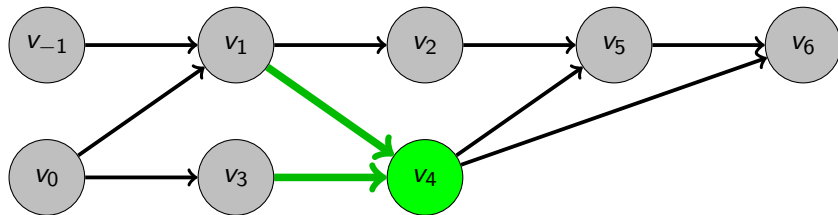
$$\dot{v}_2 = \frac{\partial (\sin(v_1))}{\partial x_1} = \cos(v_1) \dot{v}_1 = -0.6536 \cdot 1/2 = -0.3268$$

Example: Forward Mode (4/7)



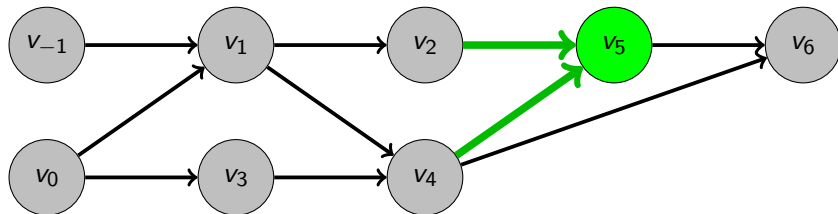
$$\dot{v}_3 = \frac{\partial (\exp(v_0))}{\partial x_1} = \exp(v_0) \dot{v}_0 = 0$$

Example: Forward Mode (5/7)



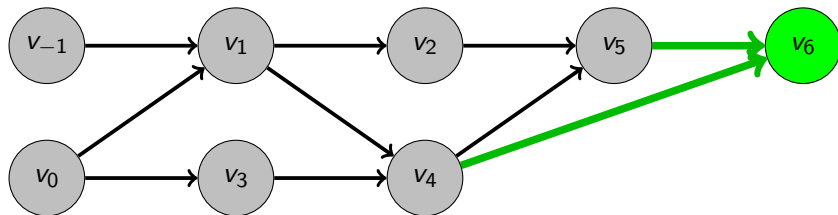
$$\dot{v}_4 = \frac{\partial (v_1 - v_3)}{\partial x_1} = \dot{v}_1 - \dot{v}_3 = 1/2$$

Example: Forward Mode (6/7)



$$\dot{v}_5 = \frac{\partial (v_2 + v_4)}{\partial x_1} = \dot{v}_2 + \dot{v}_4 = -0.3268 + 1/2 = 0.1732$$

Example: Forward Mode (7/7)



$$\begin{aligned}\dot{v}_6 &= \frac{\partial (v_5 \cdot v_4)}{\partial x_1} = v_5 \cdot \dot{v}_4 + \dot{v}_5 \cdot v_4 \\ &= -4.1459 \cdot 1/2 + 0.1732 \cdot -3.3891 \\ &= -2.6598\end{aligned}$$

Forward Mode Summary

To evaluate the gradient of y w.r.t x_2 instead of x_1 , we would repeat this entire process in the same fashion, taking derivatives with respect to x_2 along the way.

Forward-mode AD can be efficient for computing the derivatives of functions with a small number of input variables, as it only requires a single pass through the computation graph. However, it can be less efficient for functions with a large number of input variables, as **it requires separate passes through the graph for each input variable**.

In other words, we have $\mathcal{O}(n)$ calculations for n parameters.

- Same issue with the finite difference approach.

Reverse Mode

With Reverse-mode AD we will traverse the graph in a reverse direction to compute the derivatives at each node. This will facilitate the derivatives of the function with respect to all of the input variables being computed simultaneously.

We define an *adjoint* $\bar{v}_i$ as

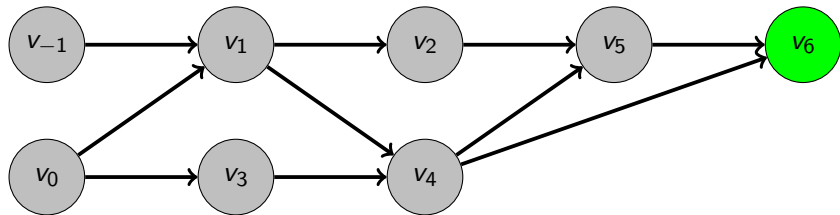
$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

Essentially, we can use the same primal trace as before and recognize that we can compute the gradient of the output with respect to any one of the intermediate variables (including the inputs) via repeated applications of the chain rule.

$$\frac{\partial y}{\partial v_i} = \sum_{k \in \text{parents}(i)} \frac{\partial y}{\partial v_k} \frac{\partial v_k}{\partial v_i}$$

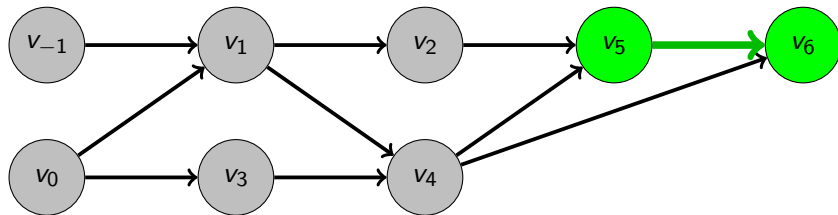
In practice, we can just *propagate backwards* the entire graph and compute everything at once.

Reverse Mode Example (1/8)



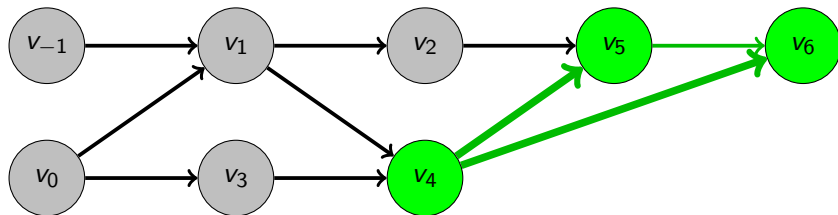
$$\bar{v}_6 = \frac{\partial y}{\partial v_6} = 1$$

Reverse Mode Example (2/8)



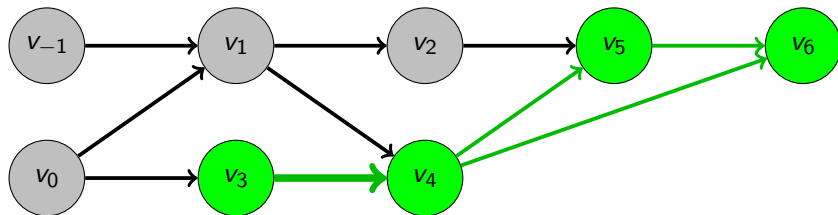
$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = \frac{\partial y}{\partial v_6} \frac{\partial v_6}{\partial v_5} = 1 \cdot \frac{\partial (v_5 \cdot v_4)}{\partial v_5} = v_4 = -3.3891$$

Reverse Mode Example (3/8)



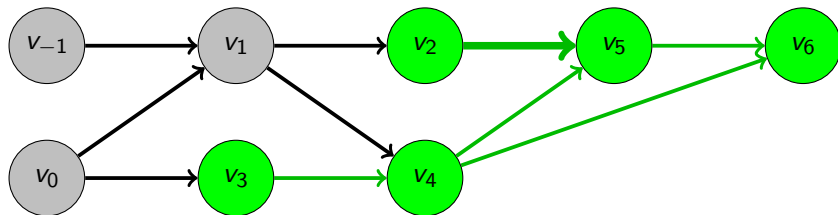
$$\begin{aligned}\bar{v}_4 &= \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_6} \frac{\partial v_6}{\partial v_4} + \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \\ &= \bar{v}_6 \cdot v_5 + \bar{v}_5 \cdot 1 \\ &= -4.1459 - 3.3891 \\ &= -7.535\end{aligned}$$

Reverse Mode Example (4/8)



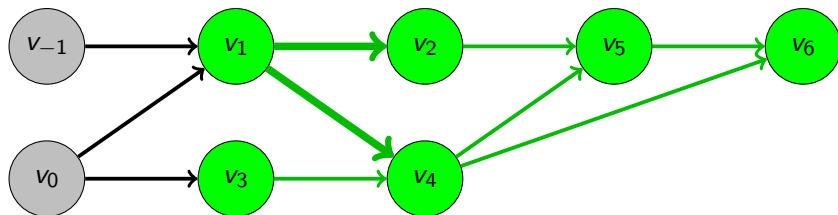
$$\bar{v}_3 = \frac{\partial y}{\partial v_3} = \frac{\partial y}{\partial v_4} \frac{\partial v_4}{\partial v_3} = \bar{v}_4 \cdot (-1) = 7.535$$

Reverse Mode Example (5/8)



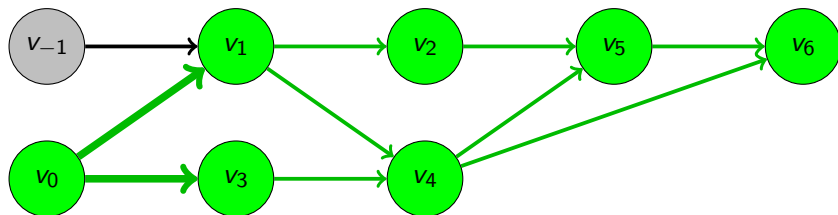
$$\bar{v}_2 = \frac{\partial y}{\partial v_2} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_2} = \bar{v}_5 \cdot 1 = -3.3891$$

Reverse Mode Example (6/8)



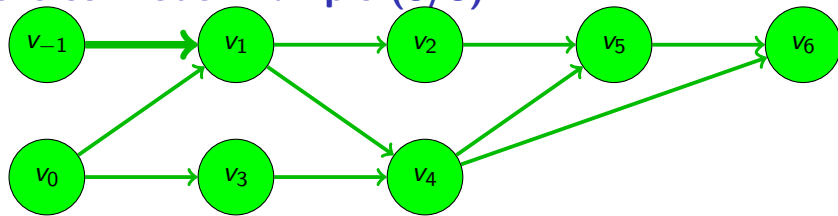
$$\begin{aligned}\bar{v}_1 &= \frac{\partial y}{\partial v_1} = \frac{\partial y}{\partial v_2} \frac{\partial v_2}{\partial v_1} + \frac{\partial y}{\partial v_4} \frac{\partial v_4}{\partial v_1} \\ &= \bar{v}_2 \cdot \cos(v_1) + \bar{v}_4 \cdot 1 \\ &= -5.3197\end{aligned}$$

Reverse Mode Example (7/8)



$$\begin{aligned}\bar{v}_0 &= \frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_1} \frac{\partial v_1}{\partial v_0} + \frac{\partial y}{\partial v_3} \frac{\partial v_3}{\partial v_0} \\ &= \bar{v}_1 \cdot -v_{-1}/v_0^2 + \bar{v}_3 \cdot \exp(v_0) \\ &= -5.3197 \cdot -2 + 7.535 \cdot 7.3891 \\ &= 66.31627\end{aligned}$$

Reverse Mode Example (8/8)



$$\begin{aligned}\bar{v}_{-1} &= \frac{\partial y}{\partial v_{-1}} = \frac{\partial y}{\partial v_1} \frac{\partial v_1}{\partial v_{-1}} \\ &= \bar{v}_1 \cdot \frac{1}{v_0} \\ &= -5.3197 \cdot \frac{1}{2} \\ &= -2.6598\end{aligned}$$

Reverse Mode Summary

To summarize the steps of reverse mode AD, we:

- 1 Run the original function forward, populating intermediate variables v_i and recording dependencies in the computational graph.
- 2 Derivatives are numerically evaluated by “back propagating” the adjoints in reverse, from output to input

Because reverse mode AD is executed by breaking down a function into elementary operations, software implementations can differentiate not only closed-form expressions, but additionally general algorithms containing control flow, loops, recursion, etc.

Reverse Mode Summary

Through this one set of steps we were able to obtain *both*.

$$\frac{\partial y}{\partial x_1} = -2.6598 \quad \text{and} \quad \frac{\partial y}{\partial x_2} = 66.31627$$

for $x_1 = 8$ and $x_2 = 2$.

Reverse-mode AD can be more efficient than forward-mode AD for computing the derivatives of functions with a large number of input variables, as it only requires a single pass through the computation graph for each output variable.

- Typically the case for neural models

The Baur-Strassen theorem states that we can compute all of the partial derivatives of a function at the same computational cost of the function itself, within a factor of 5 (Baur and Strassen 1983)

AD Summary

The technical details on *how* this algorithm is implemented is beyond our discussion.

- Operator overloading typically provides the path of implementing AD.
 - ▶ Each operation is logged onto a *tape* when the function is evaluated
 - ▶ The chain rule can then be applied by walking this tape backward.

Python has several libraries: - autograd, Chainer, PyTorch

R is a little more sparse - madness (forward mode only, 2020) - autodiff - radx

Statistical uses:

- R package glmmTMB
 - ▶ An *awesome* package for GLMMs.
- STAN

Tensorflow AD Ex $y = [\sin(x_1/x_2) + x_1/x_2 - \exp(x_2)] [x_1/x_2 - \exp(x_2)]$

```
import tensorflow as tf

def f1(x1, x2): # Define a function to be differentiated
    # Must use tf native functions- it cant trace numpy functions.
    return (tf.sin(x1/x2) + x1/x2 - tf.exp(x2))*(x1/x2 - tf.exp(x2))

# Convert the function to a TensorFlow computation graph
x1 = tf.Variable(8.0)
x2 = tf.Variable(2.0)

# Use AD to compute and print the gradients
with tf.GradientTape() as g:
    loss = f1(x1, x2)
    grads = g.gradient(loss, [x1, x2])
    print([gr.numpy() for gr in grads])

## [-2.6598396, 66.31526]
```

Tensorflow AD on multidimensional arrays

```
import tensorflow as tf
# 3 x 2 matrix of weights
w = tf.Variable(tf.random.normal((3, 2)), name='w')
b = tf.Variable(tf.zeros(2, dtype=tf.float32), name='b')
x = [[1., 2., 3.]]

with tf.GradientTape(persistent=True) as tape:
    y = x @ w + b # @ means matrix multiplication
    loss = tf.reduce_mean(y**2)
    [dl_dw, dl_db] = tape.gradient(loss, [w, b])
    # Print shapes
    print(w.shape)
    print(dl_dw.shape)

## (3, 2)
## (3, 2)
##
## WARNING:tensorflow:Calling GradientTape.gradient on a persistent
```

Optimization with Tensorflow

```
import tensorflow as tf

def f(x): return x**2 + x + 1 # min at x = -0.5

x = tf.Variable(0.0) # Start at 0

# Use SGD to minimize the loss
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)
for i in range(100):
    with tf.GradientTape() as tape:
        loss = f(x)
        grads = tape.gradient(loss, [x])

        #print(f'step {i}: The current value is {x.numpy()}, and the gradient is {grads}, and the loss is {loss}')
        _ = optimizer.apply_gradients(zip(grads, [x]))

# Print the optimal value of x
print(x.numpy())

## -0.5
```

Section 3

Derivative-free optimization

Introduction

As the name suggests, there are many methods that avoid the use of derivatives to optimize functions.

- Genetic Algorithms
- Particle Swarm Optimization
- Random optimization approaches(e.g. Luus–Jaakola)
- Simulated annealing
- And many others

Genetic Algorithms

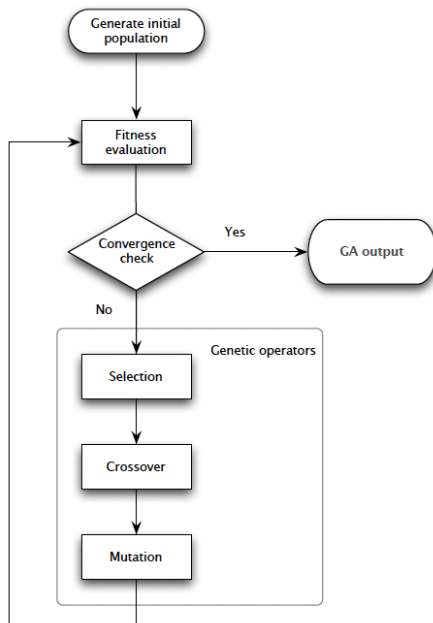
Genetic algorithms (GAs) are a type of *stochastic search* approach which attempt to optimize problems of the form

$$\Theta^* \equiv \arg \max_{\theta \in \Theta} f(\theta) = \{\theta^* \in \Theta : f(\theta^*) \geq f(\theta), \quad \forall \theta \in \Theta\}.$$

GAs proceed using an approach analagous to the evolutionary strategies of biological evolution.

- ➊ Start with random individuals
 - Could be points from a function to be optimized, a set of features, etc.
 - Each individual represents a potential solution to the problem
- ➋ Evaluate their fitness
- ➌ Select the most fit individuals for reproduction to the next generation
- ➍ Add in some random mutations
- ➎ Repeat steps 2-4 until convergence.

Flow of A Genetic Algorithm



GA Algorithm

Require: Fitness function $f(\theta_i)$

Init: Initial population of n individuals $\{\theta_1^{(0)}, \theta_2^{(0)}, \dots, \theta_n^{(0)}\}$

Init: Step counter $k = 0$

```
1 while Stopping criterion not met do
2     SELECTION: Compute probabilities  $p_i^{(k)}$  for each individual and use
        this to sample pairs of individuals who will produce the next
        generation;
3     Crossover: Combine the values of the pairs through some sort of
        average, this represents the new generation;
4     MUTATION: Add some randomness to the crossed individuals.;
5 end
```

Notes on GA Implementation

Some notes on execution

- Fitness function
 - ▶ This represents what we are minimizing, so for model fitting would be $\frac{1}{n} \sum \left(y - \hat{f}(x) \right)^2$ or similar
- Selection
 - ▶ Usually you want to oversample the population, such as sampling $2n$ individuals with replacement in order to form the pairs.
 - ▶ The probabilities $p_i^{(k)}$ should be proportional to the fitness
- Convergence
 - ▶ Practitioners often run GAs for a fixed number of generations
 - ▶ May be ideal to monitor for convergence via $\delta = \left| \theta_l^{(k)} - \theta_m^{(k-1)} \right|$ where l and m are the indices of the most fit individuals at generations k and $k - 1$.

Generally speaking, there is a fair amount of creativity in the problem solving process when applying GA

Particle Swarm Optimization

- Particles in a swarm represent potential solutions to the optimization problem
- Particles move through the search space seeking an optimal (or, at least, good enough) solution
- The particles are aware of their neighbor particles' positions
- A particle's position is adjusted according to its rate of change (velocity) and the best position found by its neighbors
- The particle velocity is based on its previous velocity (“momentum” component), its previous best location (“cognitive” component), and the best position found by its neighbors (“social” component)
- As time goes on, the swarm is focused more on the area of search space containing the best solutions

Particle Swarm Optimization Velocity Updates

Let

- θ_i - position of the i th particle (each element of θ_i corresponds to a parameter to be optimized)
- $\mathbf{v}_i$ - velocity of the i th particle
- $\mathbf{p}_i$ - personal best location of i th particle
- $\mathbf{g}_i$ - best location by particles in the neighborhood of the i th particle (note: $\mathbf{g}_i$ is not the gradient as it was above) Then, for each particle $i = 1, \dots, n$, we update the velocity and then the location according to:

$$\mathbf{v}_i \leftarrow \omega \mathbf{v}_i + \phi_1 \circ (\mathbf{p}_i - \theta_i) + \phi_2 \circ (\mathbf{g}_i - \theta_i) \quad (1)$$

$$\theta_i \leftarrow \theta_i + \mathbf{v}_i \quad (2)$$

where $\circ$ is the Hadamard (element-wise) product, ω, ϕ_1, ϕ_2 are parameters (see below), and the terms on the RHS of (1) represent “momentum”, “cognitive”, and “social”, respectively.

Particle Swarm Optimization Velocity Updates (cont)

$$\mathbf{v}_i \leftarrow \omega \mathbf{v}_i + \phi_1 \circ (\mathbf{p}_i - \boldsymbol{\theta}_i) + \phi_2 \circ (\mathbf{g}_i - \boldsymbol{\theta}_i),$$

- ω - inertial weight ($\omega > 1$ - velocities increase over time; $\omega < 1$ velocities decrease over time, converging to 0)
- $\phi_1 = c_1 \mathbf{r}_1, \phi_2 = c_2 \mathbf{r}_2$, where c_1 and c_2 are “acceleration coefficients”
 $\mathbf{r}_1, \mathbf{r}_2$ are two n -vectors containing independent samples from a $U(0, 1)$ distribution.

Particle Swarm Optimization Algorithm Details

Require: ω, c_1, c_2, n, K

Init: Initial population of n individuals $\{\theta_1^{(0)}, \theta_2^{(0)}, \dots, \theta_n^{(0)}\}$

Init: Initial swarm, $\{\theta_i : i = 1, \dots, n\}$ and initial $\{\mathbf{p}_i : i = 1, \dots, n\}$

1 **while** *Stopping criterion not met* **do**

2 **for** $i \leftarrow 1$ **to** n **do**

3 **if** $Q(\theta_i) < Q(\mathbf{p}_i)$ **then**

4 $\mathbf{p}_i \leftarrow \theta_i$;

5 $\mathbf{g}_i = \min(\mathfrak{N}(\{\mathbf{p}_j\}; i); K)$;

6 Generate ϕ_1, ϕ_2 ;

7 Update velocity using (1);

8 Update position using (2);

9 **end**

0 **end**

where $\mathfrak{N}(\{\mathbf{p}_j\}; i; K)$ finds the best $\mathbf{p}_j$ among the K nearest neighbors of the i th particle.

PSO Animation

Here, we are minimizing the function $z = x^2 + y^2$